1. SERVICES YOU SHOULD BUILD (FINAL)

☞ Keep it simple (production + deployable)

◆ Services List (ONLY 4–5)

1. **API Gateway**
    - Spring Cloud Gateway
2. **Auth Service**
    - JWT authentication
    - Roles (USER, ADMIN, HOSPITAL, VOLUNTEER)
3. **Core Service (Main Backend)**
    - User
    - Emergency
    - Hospital
    - Volunteer
    - Location logic
4. **AI Service**
    - Text → Emergency type
    - Chatbot
5. **Notification Service (optional later)**
    - Email/SMS (can add later)

⚙□ 2. WHAT TO USE (TECH STACK)

☞ Stick to this only:

- Spring Boot
- Spring Security (JWT)
- PostgreSQL
- Redis (cache + rate limit)
- WebFlux (optional)
- Kafka (later, not now)
- Docker (for deployment)

□ 3. ENTITY CLASSES (FINAL CLEAN DESIGN)

☞ Use in **Core Service**

User

```
@Entity
public class User {

    @Id
    private String id;

    private String name;
    private String phone;

    @Enumerated(EnumType.STRING)
    private Role role;

    private double latitude;
    private double longitude;
}
```

Emergency

```
@Entity
public class Emergency {

    @Id
    private String id;

    private String userId;

    @Enumerated(EnumType.STRING)
    private EmergencyType type;

    @Enumerated(EnumType.STRING)
    private EmergencyStatus status;

    private double latitude;
    private double longitude;

    private String hospitalId;
    private String volunteerId;

    private LocalDateTime createdAt;
}
```

Hospital

```
@Entity
public class Hospital {

    @Id
    private String id;

    private String name;
```

```
    private double latitude;
    private double longitude;

    private int availableBeds;
    private int icuBeds;
}
```

□ BloodStock (important)

```
@Entity
public class BloodStock {

    @Id
    private String id;

    private String hospitalId;

    private String bloodGroup;

    private int unitsAvailable;
}
```

🙋 Volunteer

```
@Entity
public class Volunteer {

    @Id
    private String id;

    private String name;

    private boolean available;

    private double latitude;
    private double longitude;
}
```

🔐 AuthUser (Auth Service)

```
@Entity
public class AuthUser {

    @Id
    private String id;

    private String email;
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

📌 ENUMS

```
public enum Role {
    USER, ADMIN, HOSPITAL, VOLUNTEER
}

public enum EmergencyType {
    CARDIAC, ACCIDENT, FIRE
}

public enum EmergencyStatus {
    CREATED, ASSIGNED, IN_PROGRESS, COMPLETED
}
```

□ 4. SYSTEM DESIGN (ENGLISH – CLEAN)

◆ High-Level Architecture

This system follows a **microservices + modular monolith hybrid architecture**.

- API Gateway acts as a single entry point.
- Auth Service handles authentication and authorization using JWT.
- Core Service contains all business logic (users, emergencies, hospitals, volunteers).
- AI Service processes user input and detects emergency type.
- Redis is used for caching and rate limiting.

◆ Flow of Emergency Request

1. User sends request → API Gateway
2. Gateway validates JWT → forwards to Core Service
3. Core Service:
    - saves emergency
    - calls AI Service to detect type
    - finds nearest hospital (using location)
    - assigns volunteer
4. Response returned to user
5. (Optional) Notification sent asynchronously

◆ Communication

- REST APIs (initially)
- Kafka (later for async events like notifications)

◆ Database Design

- PostgreSQL → main data (users, emergencies, hospitals)
- Redis → caching (locations, hospitals)

◆ Scalability Strategy

- Start with single Core Service
- Later split into:
  - Emergency Service
  - Hospital Service

◆ Security

- JWT-based authentication
- Role-based access control
- API Gateway filter for validation

◆ Real-Time Features (Optional)

- WebSocket for:
  - live tracking
  - emergency status updates

💣 FINAL SUMMARY (VERY IMPORTANT)

👉 Build only this:

- Auth Service ✅
- Core Service ✅

- AI Service ✅
- Gateway ✅

👉 Use:

- PostgreSQL
- Redis
- JWT

👉 Avoid now:

- ❌ Too many microservices
- ❌ RabbitMQ
- ❌ Over engineering